

## I. Database management system :- C Quick Notes

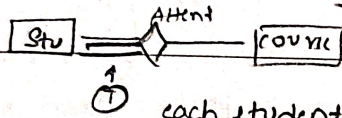
\* collection of software that provides a quick way to access & modify data.

① Evolution of DBMS:- [file based / RDBMS / NoSQL] \* Schema:- Dp of the structure of DB.  
 (structured/rate/strict) (Rows Table)  $\rightarrow$  (flexible, fast, scalable) (Horiz)

② Entity / Relationship / Attribute  
 (Noun) (Verb) (characteristic)  
 □ ◇

③ ER diagrams:- (unary / Binary / n-ary) (most common) \* ④ Cardinality:- one-one / one-many / m-n  
 □-□ □-□-□ □<sup>1</sup>-□<sup>1</sup> □<sup>1</sup>-□<sup>N</sup> □<sup>m</sup>-□<sup>n</sup>

⑤ Total participation:- Every entity of one side participates in relationship.



\* Key:- uniquely identified element

⑥ Weak Entity:- don't have own key:- (class)  $\rightarrow$  (relation)  $\rightarrow$  WE  
 (ECE) contains (R)

⑦ Architecture:- (T1) (T2) (T3)  
 (All in one place) (Client  $\rightarrow$  DBMS) (Client  $\rightarrow$  App server  $\rightarrow$  DB)  
 (directly) (server)

⑧ Key:- C/S (unique), never repeat, can be many, minimal

PK (only one)

1K  $\subset$  C/S (set of Attribute that uniquely identifies (C/E No, Name))

Note:- Closure of A  $\rightarrow$  P1 (A, B, C, D)  $\Rightarrow$  A  $\rightarrow$  B, B  $\rightarrow$  C, C  $\rightarrow$  D

• Reflexivity:- AB  $\rightarrow$  A (Y  $\subset$  X) X  $\rightarrow$  Y At = {A, B, C, D}

• Aug:- X  $\rightarrow$  Y, XZ  $\rightarrow$  YZ

• No of superkeys:-  $2^{(n-k)}$

• Tr:- X  $\rightarrow$  Y, Y  $\rightarrow$  Z, X  $\rightarrow$  Z

• no of attribute = n = 5

• candi'date key size = 15 = 2

⑨ Foreign Key:- key which refers to some primary key of different or same table. (REFERENCES)

$2^3 = 8 //$

ON CASCADE DELETE (delete) on both (F.R / P.K)  
 (Referential Integrity)

## II. Normalization:- Data redundancy, Integrity ↑, (Updation, deletion, insertion Anomaly) ↓

① Functional dependency:-  $(A \rightarrow B)$  (Dept ID  $\rightarrow$  Dept)  $\rightarrow$  {separate table & eliminate (dep Name column)}

$(A) \rightarrow (B)$   
 (name)  $\rightarrow$  (name)

② 1<sup>st</sup> Normal form:- (No multivalued attribute)

③ 2<sup>nd</sup> Normal form:- (2NF + No partial dep, only full dep.)

eg:-  $Student(A) \xrightarrow{\text{primary}} C, B \xrightarrow{\text{secondary}} C$

$(AB) \rightarrow (C)$  (Non-uniform)  
 $B \rightarrow C$  (partial dep) |  $A \rightarrow C$

④ 3<sup>rd</sup> Normal form:- (2NF + No Transitive dep)

(No - Non prime should determine Non - prime) eg:-

$Topper \rightarrow Friend \rightarrow f_2$   
 (PK) (NP) (NP)

BCNF:- [3<sup>rd</sup> NF + (LHS  $\rightarrow$  CK or PK)]

$X \rightarrow Y$   
 (CK or PK)

\* Note:- 4<sup>th</sup>  $\rightarrow$  BCNF + No multivalued dep  
 NT  
 Varun  $\rightarrow$  3 phone + 2 email  
 (5<sup>th</sup> NF:- 4<sup>th</sup> + q comb)  
 (6<sup>th</sup> NF:- 4<sup>th</sup> + 5<sup>th</sup> + load decompose)

### Problems:- (Tricks)

① Irreducible set (Minimal cover):- is RHS  $\rightarrow$  single value

ii) Redundancy  $\rightarrow$  pick a relation, get closure of it eg:-  $(A \rightarrow \text{List everything})$   
 • if the closure has something of that only  $\rightarrow$  give A.  
 • relation  $\rightarrow$  redundant or else no.

• If any relation  $\rightarrow$  redundant should not include it while checking closure.

iii) (LHS)  $\rightarrow$  single value:-  $AC \rightarrow D \rightarrow A^+ =$  If "C" comes remove C or vice versa

② check Highest NF? is find All CK  $\rightarrow$  (closure)  $\rightarrow$  replace with RHS of CK

ii) write all prime attribute iv) FD:-

iii) Non-prime:- 2<sup>nd</sup> NF:- (LHS  $\rightarrow$  CK or PK)  $\rightarrow$  3<sup>rd</sup>:- LHS  $\rightarrow$  RHS (P) (P)

2<sup>nd</sup> NF:- LHS  $\rightarrow$  proper subset of CK

RHS  $\rightarrow$  NP  $\rightarrow$  (X)

1<sup>st</sup> NF:- Multivalued

Note: proper subset v/s subset (entire)  
 (partial)  
 BCNF  $\rightarrow$  0% (redundancy)

decompose:- (data preserved / lossless decomposition)

R (A B C D E F)

P1 / (A B E F)

P2

CD (problem)

(looks for something common, if not they will never join)  $\rightarrow$  look for common (C) (no problem) (R)

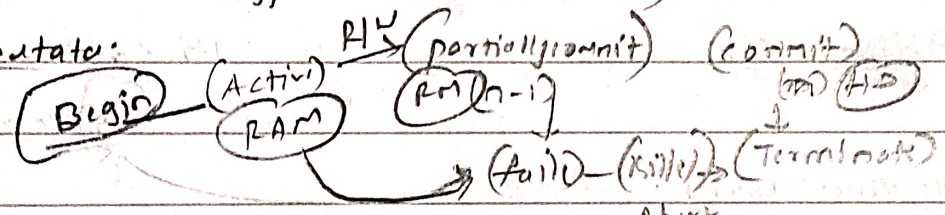
### III. Transactions:- (Movs $\rightarrow$ set of operations)

Read  $\rightarrow$  commit (changes permanently saved to DB)  
 Write

① ACID:- (Atomicity) / (Consistency) / (Isolation) / (Durability)

Rollbacks:- ( $T_1 \rightarrow$  fails  $\rightarrow$  any point  $\rightarrow$  entire  $T_1$  fails)

Transaction state:

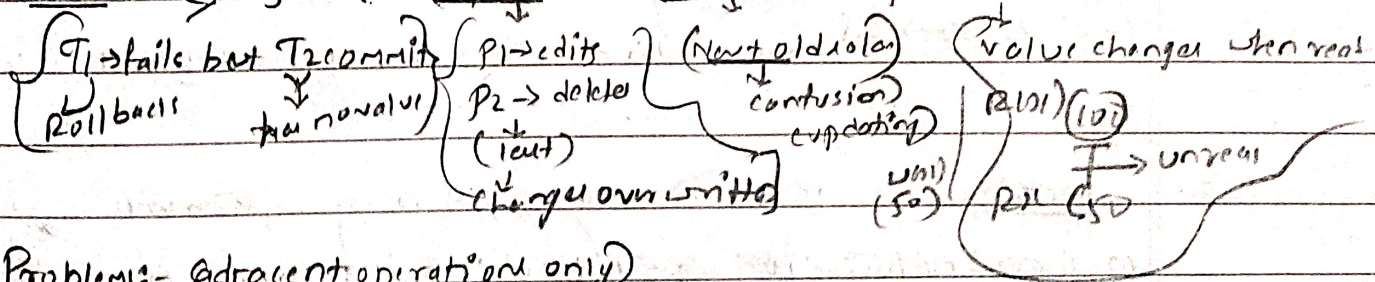


② schedule:- chronological execution sequence of multiple Transactions

③ multiple Transactions

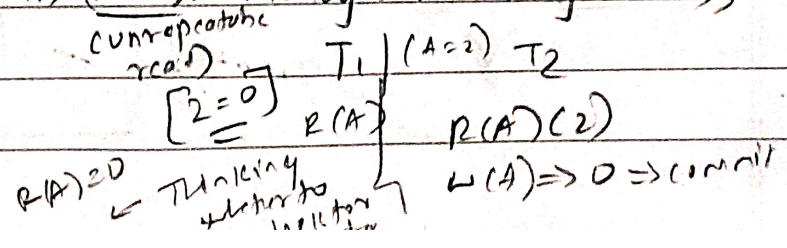
[one after other] [many @ same time]  $\rightarrow$  p $\uparrow$ , Throughput  $\uparrow$  [No of Trans / Time]

Problems:- (Dirty read / lost update / Incorrect / unrepeatable read)



Problems:- adjacent operations only

i) (R-R) :- (DIRTY read) ii) (R-W) :- lets say u r booking bus (2)



iii) Irrecoverable vs recoverable schedule  $\rightarrow$  Cascading vs cascadeless  
 (due to occurrence of 1 event, multiple events are automatically occurring.)  
 vs W-W: (adjacent write)

iv) Serializability:- make all schedule  $\rightarrow$  serial (clone it)  
 ① Conflict Equivalent:-  $S \equiv S' \{ R(A) W(A), W(A) R(A), W(A) W(A) \}$   
 (conflict pairs)  
 $\rightarrow$  Adjacent pairs swap  $\rightarrow$  (Non-conflict)  
 $\rightarrow$  conflict  $\rightarrow$  No change leave it there.

$S \equiv S' \Rightarrow$  serializable

② Conflict serializability:-

Precedence graph:-  
 A) Check conflict pairs in other  
 Transactions & draw edge  $\{ (R, W), (W, R), (W, W) \}$   
 (No loop/cycle  $\rightarrow$  conflict  $\rightarrow$  Serial)  
 (Indegree = 0)  $T_2 \rightarrow T_3 \rightarrow T_1$

③ View serializable:-

|        | T1     | T2     | T3     |  | T1     | T2     | T3     |
|--------|--------|--------|--------|--|--------|--------|--------|
| $R(A)$ | $R(A)$ |        |        |  | $R(A)$ |        |        |
| $W(A)$ | $W(A)$ | $W(A)$ |        |  |        | $W(A)$ |        |
| $W(A)$ |        |        | $W(A)$ |  | $W(A)$ |        | $W(A)$ |

- $\rightarrow$  If schedule produces same final result as some serial schedule.
  - $\rightarrow$  final op matters
  - $\rightarrow$  Every conflict  $\rightarrow$  View but  
 $\{ \text{e.g. } X \rightarrow CS \}$
- (then we interchange operation)  $\leftarrow$  (Conflict)  $\otimes$

v) locking:-

① shared exclusive locking:- (Serializable | Irrecoverable deadlock | starvation)  
 (read only)  $\{ W | R \text{ both} \}$

grant

|   | S | X |
|---|---|---|
| S | ✓ | X |
| X | X | X |

② 2PL locking:-   
 Growing → locks acquired & no locks are released  
 Shrinking → locks released & no locks → acquire

Adv:- fully serializable → (once growing then only shrinking)  
 → (Dead State Error) → problems. Note:- locks point:- point where a transaction acquires its last lock after that

③ strict 2PL:-  $2PL + X(A) \rightarrow$  should hold: it only releases until commit.

2PL guarantee:-  $2PL + (S(A) + X(A)) \rightarrow$  should hold until commit

Adv → Irrecoverability problem solved  
(Cascading rollback solved)

→ DL + starvation

Note:- conservative:- declare all locks before & DL → remove

④ Timestamp ordering:- (unique value) <sup>assigned</sup> tells order in which enters system.

(RTS) → latest transaction no performed read successfully

(WTS) —||—||—||—||— write —||—  $TS(T_i)$

Rule:- (which comes first → it completes first) →  $(T_1 \rightarrow T_2)$  always

i)  $T_i$  issues a R(A):- if  $WTS(A) > TS(T_i) \rightarrow$  Rollback  $T_i$

otherwise:- set  $RTS(A) = \max\{RTS(A), TS(T_i)\}$

ii)  $T_i$  issues W(A):- if  $RTS(A) > TS(T_i) \rightarrow$  Rb  $T_i$   
 $WTS(A) > TS(T_i)$

otherwise set  $WTS(A) = TS(T_i)$

Problem:- • TS → given of all Transactions (Polu)  
 • RTS / WTS → given (also update)  
 ⇒ Rollback = ? → which operation.

VI. Indexing:- (CPU) → (RAM) → (HD) → (Block) → I/O Time reduce → Indexing

Avg Time complexity (unordered) :- No. of records in each block  
 \* (Best case → 1 / Worst → eg: last)  $(N)$  Tot No. of blocks req. →  $(N)$

\* Unordered:  $N/2 \rightarrow 12/2$  / ordered:  $\log_2 N$  :- 12/11 → eg only smaller than unordered

① With Indexing:-

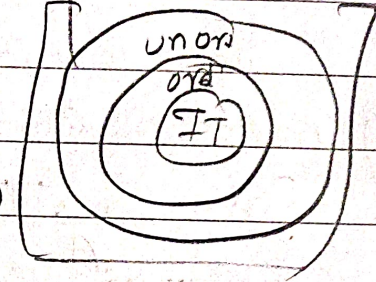
[IT size = block size HD]

It will yield much smaller avg time complexity.

| key value | points |
|-----------|--------|
|           |        |
|           |        |
|           |        |

IT

dense → each value has  
sparse → only few imp.



② Types:- (Primary / clustered / secondary)

ordered file

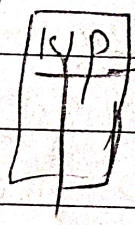
|   |     |
|---|-----|
| p | c   |
| g | see |

unordered

key Non-key  
(unique) (repetable)

Note:-  $\log_2 N$  :- No. of times a dataset of size divided into half before it is reduced to single element.

③ Primary Index:-  $(0 + 1) \rightarrow$  sparse



$(\log_2 N + 1)$

→ No. of blocks in IT.

④ Clustered:-  $(0 + N/2) \rightarrow$  There is chance for same entries in multiple blocks.

$(1 \rightarrow \text{blk } 1), (1 \rightarrow \text{blk } 2 \rightarrow 3) \rightarrow$  due to Non-key  $\rightarrow (\log_2 N + 1 + 1 \rightarrow \text{Block pointer})$

(sparse) 1 only can be used as clustered Index.

⑤ Secondary:- eg: when query is for name, when ID is 1<sup>st</sup> Index, it fails cause name can have multiple IDs, which can get confused

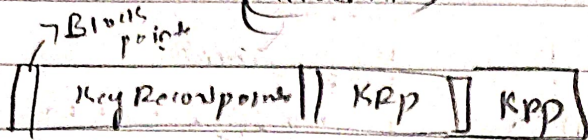
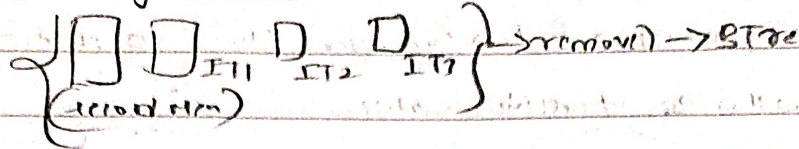
$(IT :- \log_2 N + 1) \rightarrow$  Dense

$(0 + 1)$

$(UNO + N/2) \rightarrow$  Intermediate (IT) or block record pointers  $\rightarrow (\log_2 N + 1 + 1)$   
 $(0 + 1)$

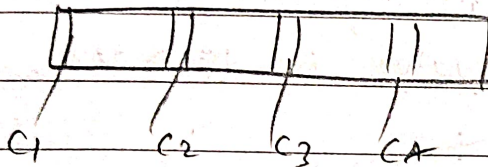
### ③ B-Tree:- Multilevel Dynamic Index:-

Why needed?

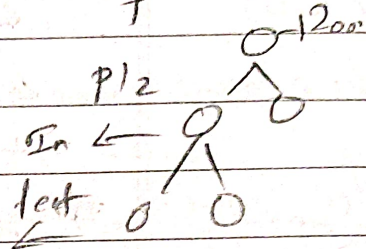


(Order  $\Rightarrow p = \text{Max no of children}$ )  
 ( $p = 4$ )

( $1 \text{ Key} = p - 1$ ) ( $Rp = p - 1$ )

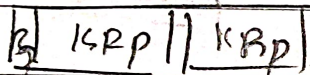


[problem?]

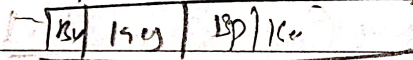


Key size = 10 byte, Block size = 128, data ptr  $\rightarrow$  8B,  $Bp = 5$  order BTree

$$(n \times Bp + (n - 1) \text{ Key} + (n - 1) Rp \leq \text{Block size})$$



①  $(n \times Bp) + (n - 1) \text{ Key} \leq Bp \Rightarrow \text{BTree} \rightarrow \text{Non-leaf Internal}$



leaf [KeyRp | KeyRp]

$$(L \text{ Key} + Rp) + Bp \leq Bp$$

|           | BTree                | B+Tree                                |
|-----------|----------------------|---------------------------------------|
| Data      | stored in Int + leaf | leaf                                  |
| Searching | slow                 | leaf $\rightarrow$ Searching/deletion |
| Deletion  | complex              |                                       |
| Linked    | X                    | ✓                                     |
| Files     |                      |                                       |
| Index     | not linked together  | link                                  |

VII

Recovery:- (log based)

File → store → (action)

Deferred

If Transaction fails → (recovery) check to modify

→ we only store New value.

recovery:

No undo /

(Redo)

$\langle T_1, \text{start} \rangle$

$\langle T_1, A, 200 \rangle$

$\langle T_1, B, 400 \rangle$

$\langle T_1, \text{commit} \rangle$

Redo

but No commit

↓  
(Rollback)

(Immediate)

It does not wait for commit to modify value in DB, as in case of

Deferred  $\rightarrow W(A) \Rightarrow 200$ , value in DB  $\rightarrow 200$ .

$\langle T_1, \text{start} \rangle$

$\langle T_1, A, 200, 200 \rangle$

$\langle T_1, B, 200, 400 \rangle$

$\langle T_1, \text{commit} \rangle$

Redo

Not commit

↓  
Undo

CN:- collection of interconnected computing devices that communicate with each other to share data, resources etc.

① protocols:- set of rule used by 2 device to communicate.

② Open system interconnect (Theoretical)  
(TCP/IP) must (A)  
A → provide Network service to user App (Browser, email)  
P → Translate, compress, encrypt data  
S → establish, manages, terminates communication

segment → T → reliable data delivery with error checking & flow control (End to End)  
(packet) → N → finds best path & routes data → IP (logical add) → unreliable / reliable  
unreliable connectionless → DL → Transfers data b/w directly connected devices NA (Add) (Device to Device)  
frame → P → Transmits raw bits (0/1) → cables, fiber, wireless  
add header ↓ (S) (communication only) (R) (removed header)  
Through respective layers

④ Transmission delay:-

$$(T_D) = \frac{L}{BW} \rightarrow \text{min data can be sent/sec}$$

(E) → DC Time Taken by sender to put data on channel

Propagation delay:- Time Taken by packet to reach receiver completely.  
(T\_P) → (distance/velocity)

Queueing delay:- (P) → waiting time → (R) → Processing delay:- processing time

②. Stop & wait Error & Flow control:- happens in (T) & (DL)  
(Automatic repeat request) → R & W

Based on sliding window { Global SN  
Selective repeat

handle → flow control

lost data, ACK

delayed data, ACK

